
Exercises: Advanced Lesson 6

Your Name: Mikey Ferguson

Your Math Class: MATH 7013 - Advanced Mathematical Modeling

Note: Some of the following exercises are expressed in every day mathematical notation rather than proper *Mathematica* notation. You MUST convert every day notation into *Mathematica* notation for your code to work properly. For example:

Equations must be changed from "=" to "==" to work correctly (i.e. $x^2 == 2x$ not $x^2 = 2x$).

Functions must be expressed as $f[x]$ and not $f(x)$ to work correctly.

Functions must be defined using "_" to work correctly (i.e. $f[x_] = 2x^2 + 1$ and not $f[x] = 2x^2 + 1$).

Mathematica functions must start with a capital (i.e. $\text{Sin}[x]$ not $\text{sin}[x]$).

Your functions should start with a lower case letter (i.e. $f[x]$ not $F[x]$).

Special symbols must be entered correctly (i.e. e from the palette and not e).

These exercises begin your training in making this conversion automatically.

Remember to clear all variables that you will use below: `Clear[ ... ]`

1: Explain why the following code gives unexpected results.

```
x = 2  
 $\int x^2 dx$ 
```

```
Clear[x];
```

```
x = 2;
```

```
 $\int x^2 dx$ 
```

x had already been given the value 2, so by the time the integral was read it said integrate 4 with respect to 2. The integration variable itself had turned into a number, which is meaningless, so the integral was left unevaluated.

2: Explain why the following code gives unexpected results.

```
x := 2
 $\int x^2 dx$ 
```

```
Clear[x];
```

```
x := 2;
```

```
 $\int x^2 dx$ 
```

The same thing happens. Using := only delays the evaluation of x; x still becomes 2 as soon as it is used, so the integration variable is again the number 2.

3: Explain why the following code gives unexpected results.

```
x = 2;
f1[x_] = x^2;
f2[x_] := x^2;
Plot[{f1[z], f2[z]}, {z, 0, 3}, PlotStyle -> {Thickness[0.01]}];
```

```
Clear[x, f1, f2];
```

```
x = 2;
```

```
f1[x_] = x^2;
```

```
f2[x_] := x^2;
```

```
?f1
```

```
?f2
```

```
{f1[z], f2[z]}
```

```
Plot[{f1[z], f2[z]}, {z, 0, 3}, PlotStyle -> {Thickness[0.01]}]
```

x held the value 2 when f1 was defined with =, so the right-hand side x^2 was evaluated at once to 4, and f1 stores 4 for every argument. f2 was defined with :=, so it keeps x^2 and squares whatever it is given. That is why f1[z] is 4 and f2[z] is z^2, and the plot shows one flat line and one parabola.

4: Explain why the following code gives unexpected results.

```
Clear[x, y]
x^2 /. x -> y
 $\int x^2 dy$ 
```

```
Clear[x, y];
```

```
x^2 /. x -> y
```

```
 $\int x^2 dy$ 
```

The replacement itself is fine and gives y^2 . The integral is the surprise: it asks for the integral of x^2 with respect to y , and x does not depend on y , so the answer is $x^2 y$. The replacement rule changed nothing permanently, so x is still x on the next line.

5: Explain the results of the following code:

```
Clear [x, y, z]
x = x /. x → y
 $\int x^2 dy$ 
```

```
Clear[x, y, z];
x = x /. x → y
```

```
 $\int x^2 dy$ 
```

This line assigns the result of the replacement, so x now genuinely holds y . The next line therefore integrates y^2 with respect to y and gives $y^3/3$.

6: Explain the results of the following code:

```
Clear [x, y, z]
x = z /. x → y
 $\int x^2 dy$ 
```

```
Clear[x, y, z];
x = z /. x → y
```

```
 $\int x^2 dy$ 
```

Here the rule does not apply, because the expression being replaced is z and the rule looks for x . So x is assigned z , and integrating x^2 , which is now z^2 , with respect to y gives $y z^2$.

7: Explain why the following code does NOT yield: $\text{Log}[2 \pi y]$

```
Clear [x,y]
Sin[x] /. {Sin[x] → Log[x], x → 2  $\pi$  y}
```

How would you modify the replacement rule to yield the desired result?

```
Clear[x, y];
Sin[x] /. {Sin[x] → Log[x], x → 2  $\pi$  y}
Sin[x] //. {Sin[x] → Log[x], x → 2  $\pi$  y}
Sin[x] /. Sin[x] → Log[2  $\pi$  y]
```

`/.` makes a single pass. It matches `Sin[x]` and replaces it with `Log[x]`, and the result is not scanned again, so the second rule never reaches the x inside. `//.` repeats the rules until nothing changes, and that gives `Log[2  $\pi$  y]`.

8: Explain how the following code works:

```
Clear [f,x,y,z]
f[x_] := x /. {y → Cos[y], z → Sin[z]}
```

```
f[x]
f[x + y + z]
```

```
Clear[f, x, y, z];
f[x_] := x /. {y → Cos[y], z → Sin[z]}
```

```
f[x]
f[x + y + z]
```

The definition says: take whatever is handed in and apply the two rules to it. `f[x]` gives back `x`, because there is no `y` or `z` in it. `f[x + y + z]` gives `x + Cos[y] + Sin[z]`, because the rules find the `y` and the `z` inside the sum.

9: Explain the purpose of the replacement rules: `Frame → False`, `Frame → True`

```
Clear[x]
Plot[Sin[x], {x, 0, 2π}, Frame → False]
Plot[Sin[x], {x, 0, 2π}, Frame → True]
```

```
Clear[x];
Plot[Sin[x], {x, 0, 2π}, Frame → False]
Plot[Sin[x], {x, 0, 2π}, Frame → True]
```

`Frame → True` and `Frame → False` are options, and an option is written as a replacement rule. `Plot` reads that rule to decide whether to draw a box around the plot.

10: Explain how the following code works:

```
Clear[x, plot1, plot2]
plot1 = Plot[Sin[x], {x, 0, 2π}, Frame → True]
?plot1
plot2 = plot1 /. (Frame → {{True, True}, {True, True}}) → (Frame → {{False, True}, {False, True}})
?plot2
Show[plot2]
```

```
Clear[x, plot1, plot2];
plot1 = Plot[Sin[x], {x, 0, 2π}, Frame → True]
?plot1
Frame /. Options[plot1]
plot2 =
  plot1 /. (Frame → {{True, True}, {True, True}}) → Frame → {{False, True}, {False, True}};
Frame /. Options[plot2]
Show[plot2]
```

A finished plot is just an expression: a Graphics object holding the drawing and a list of option rules. Applying a replacement rule to it rewrites the Frame option in place, and the result is a new, valid plot which Show displays. Nothing is recomputed; the picture is edited the same way any other expression is.

11: Part 1: Solve the ODE: $y''[t] + y[t] = 0$ to get a replacement rule for $y[t]$ using DSolve.

Part 2: Use a list of replacement rules to replace $C[1]$ with 4 and $C[2]$ with 7 in the replacement rule for $y[t]$.

Part 3: Use the solution replacement rule and substitute back into the ODE to check the solution (You MUST use a procedure similar to the one I used above when I used the Solve command, however this example is harder than the one above).

Hint: How does $\partial_{t,t} (y[t] \rightarrow f[t])$ evaluate?

Part 4: Use replacement rules to extract the solution $y[t]$ from its replacement rule generated in Part 2 and use this to plot the solution for $0 \leq t \leq 2\pi$.

```
Clear[y, t, sol, sol2];
sol = DSolve[y''[t] + y[t] == 0, y[t], t]
sol2 = sol /. {C[1] -> 4, C[2] -> 7}
rule = First[sol2]
drule =  $\partial_{\{t,2\}}$  rule
Simplify[y''[t] + y[t] /. drule /. rule]
ysol = y[t] /. rule
Plot[ysol, {t, 0, 2  $\pi$ }]
```